
vmadvise

A Userspace Residency-Hinting Library for GPU Memory

Sushant Padha
24B1057

Koduru Tejeswar
24B0918

Mentor: Prof. Purushottam Kulkarni
Department of Computer Science and Engineering, IIT Bombay

Abstract

Contents

1	Introduction	2
2	UVM vs. VMM	2
2.1	Why NVIDIA introduced each	2
2.2	Representative workloads	2
2.3	Capabilities VMM provides that UVM does not	2
2.4	Experiments to isolate the difference	3
3	PyTorch Caching Allocator Study	3
4	Library Design	3
4.1	Desired Features	3
4.2	Use Cases	3
4.3	API	3
4.4	Design	3
5	Experiments	3
6	Conclusion	3

1 Introduction

2 UVM vs. VMM

CUDA provides two distinct mechanisms for managing GPU memory beyond `cudaMalloc`: Unified Memory (UVM, `cudaMallocManaged`) and the Virtual Memory Management API (VMM, `cuMemCreate`/`cuMemAddressReserve`/`cuMemMap`/`cuMemSetAccess`). These mechanisms address different problems.

- Unified Memory: <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/unified-memory.html>
- Virtual Memory Management: <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/virtual-memory-management.html>

2.1 Why NVIDIA introduced each

UVM addresses a programmer-burden problem. Prior to its introduction, every host/device pointer pair required explicit `cudaMemcpy` calls to move data between host and device code. `cudaMallocManaged` returns a single pointer valid on both host and device, and the driver migrates pages on demand as they are accessed. `cudaMemAdvise` and `cudaMemPrefetchAsync` exist to improve performance beyond this default behavior by supplying hints to the driver.

VMM addresses a different problem: it gives an application that already knows its memory placement requirements a way to express them explicitly. The CUDA Programming Guide motivates the API through a specific limitation of the classic model: `cudaEnablePeerAccess` maps all of a process's past and future allocations to a peer device, so sharing even a small number of buffers incurs mapping overhead on every allocation the process ever makes. VMM separates `cudaMalloc` into two steps. `cuMemAddressReserve` reserves a virtual address range with no physical backing; `cuMemCreate`, `cuMemMap`, and `cuMemSetAccess` then bind physical memory to that range under explicit, per-mapping access control.

2.2 Representative workloads

UVM is well suited to workloads with irregular, data-dependent access patterns, where a hand-written copy schedule is impractical: graph algorithms such as breadth-first search or PageRank on graphs too large or too irregular to predict which region will be needed next, out-of-core algorithms whose working set exceeds device memory, and early-stage kernel development where correctness matters more than placement.

VMM is well suited to workloads that already know their placement requirements and are penalized by not being able to express them: custom allocators requiring in-place growth without copying data, key-value cache managers in large language model inference, and multi-GPU communication libraries such as NCCL and NVSHMEM, which share specific buffers across devices rather than mapping entire address spaces.

	UVM	VMM
Core primitive	One pointer, driver migrates pages on fault	Reserve (VA), create (physical), map, and set-access, all explicit
Residency control	Advisory hints only	Deterministic: resident exactly where last mapped
Growth	Fixed size at allocation	Chunked, non-contiguous physical backing behind one contiguous virtual range
Cross-process sharing	Not a first-class primitive	First-class: handles are exportable and importable

2.3 Capabilities VMM provides that UVM does not

- **Non-contiguous physical backing behind one contiguous virtual address range.** A UVM allocation has a single, fixed physical footprint determined at call time. VMM allows separately allocated chunks to be mapped into one contiguous virtual range, which is the mechanism PyTorch's `expandable_segments` allocator relies on.
- **Deterministic residency rather than a hint.** `cudaMemAdvise` is advisory and the driver may disregard it. `cuMemMap` and `cuMemUnmap` are not requests: once `cuMemMap` returns, the mapping exists.

This property is what the `cuda_remap` primitive in this project depends on, since it requires the caller to know precisely when a backing swap between device and host memory has completed.

- **Live, cross-process re-sharing of a single physical allocation.** A VMM handle can be mapped into more than one process's address space, and that mapping can change while other processes hold their own. UVM has no equivalent mechanism.
- **No page-fault safety net.** There is no fault handler for VMM-mapped ranges: a kernel accessing a virtual address that has been unmapped via `cuMemUnmap` terminates the process rather than faulting and recovering. UVM's entire mechanism is, by contrast, built around a fault handler. Any library built on VMM must therefore guarantee correctness of in-flight remap operations itself.

2.4 Experiments to isolate the difference

Establishing whether VMM's additional control is worthwhile on this hardware, rather than UVM's driver heuristics already being sufficient, requires direct measurement. The first experiment is as follows.

Interrupt-driven paging versus advised prefetch with remap. A working set larger than device memory will be driven two ways: first with UVM and no hints, relying entirely on fault-driven migration; second with VMM, issuing `cuda_remap` ahead of the kernel that requires the data. Per-kernel stall time and total wall-clock time will be measured for each.

3 PyTorch Caching Allocator Study

4 Library Design

4.1 Desired Features

4.2 Use Cases

4.3 API

4.4 Design

5 Experiments

6 Conclusion

References